Selenium + BDD Retail Automation Test Report

Participant: Justin Philip
Date: 29th Jun 2026
GitHub Repository:

- Automation Project:)

---

1. Objective & User Story

Objective

The objective of this automation is to validate end-to-end user workflows using Selenium with a maintainable automation framework.

The framework aims to:

- Automate critical UI flows
- Ensure application stability across builds
- Provide reliable reporting and execution evidence

User Story

As a customer, I want to search for a product and validate its details before viewing the product page.

---

2. Environment & Setup

Layer	Tools / Configuration
Language	Java
Build Tool	Maven
UI Automation	Selenium WebDriver
BDD	Cucumber
Test Framework	JUnit 5
Reporting	Allure / Extent

Layer	Tools / Configuration
Browser	Chrome / Edge
Execution Mode	Headless / UI Mode

Prerequisites

Install dependencies

mvn clean install

Run tests

mvn test

---

3. Framework Design

3.1 Page Object Model

The framework follows the Page Object Model (POM) design pattern:

- Each page contains locators and actions
- Test logic is separated from UI interactions
- Improves maintainability and reusability

3.2 Framework Components

- **Pages:** BasePage, HomePage, ProductListingPage, ActualProductPage
- **BDD:** World, Hooks, CucumberRunningHere, Journey Steps
- **Test Layer:** Step definitions / test classes
- **Support:** Config, EnvCheck, DriverFactory
- **Utilities:** ProductViewCard

🟡 EDIT THIS: (Change based on your actual class names)

3.3 Hooks

- @Before → Browser initialization
- @After → Browser cleanup
- Failure handling → Screenshot capture

---

4. Test Scenario Covered

Flow

1. Navigate to https://www.amazon.in
2. Search for "wireless mouse"
3. Click Search.
4. Verify search results page loads.
5. Capture all displayed product names.
6. Select any product dynamically from the first page.
7. Validate:
    a. Product title is displayed.
    b. Product price is available.
    c. Product rating (if available).
8. Open the product.
9. Verify product page title contains the product name.

---

5. Detailed Test Scenario

Scenario 1

Scenario Name: Searching Product and validating results
Category: Smoke

Steps:

1. Navigate to https://www.amazon.in
2. Search for "wireless mouse"
3. Click Search.
4. Verify search results page loads.

5. Capture all displayed product names.
6. Select any product dynamically from the first page.
7. Validate:
a. Product title is displayed.
b. Product price is available.
c. Product rating (if available).
8. Open the product.
9. Verify product page title contains the product name.

Expected Result:

- User should verify the product details successfully

Actual Result:

- User verifies the product details successfully

Status: Pass

---

6. Execution Details

Run Commands

Run all tests

mvn test

Run smoke tests

mvn test -Dcucumber.filter.tags="@smoke"

Run regression tests

mvn test -Dcucumber.filter.tags="@regression"

Run negative tests

mvn test -Dcucumber.filter.tags="@negative"

What It Proves

- Tests execute successfully via Maven
- Tag-based execution works correctly
- Framework supports selective execution

---

7. Results

Execution Summary

Scenario	Category	Result
Test 1	Smoke	Pass

Overall Result

- Total Scenarios: 1
- Passed: 1
- Failed: 0

---

8. Reporting & Evidence

The framework supports:

- Test execution logs
- Allure / Extent reports are available
- Screenshot capture on failure

Screenshot Handling

- Screenshots are automatically captured when a test fails
- Attached to reports for debugging

---

10. Environment Information

Example:

- Browser: Chrome
- OS: Windows

- Java Version: 22

---

11. Issues & Limitations

- Limited test coverage in current implementation
- Some scenarios may require additional validation
- Reporting tools may not be fully configured

---

12. Best Practices Followed

- Page Object Model implemented
- Reusable components used
- No hard waits (uses explicit waits)
- Clean separation of test and page logic

---

13. Conclusion

The Selenium automation framework successfully validates core user journeys.

The framework demonstrates:

- Stable execution
- Maintainable structure
- Scalability for future enhancements

However, additional test scenarios and reporting improvements are recommended for production-level quality.
